



Facultad de Ingeniería
Departamento de Ingeniería en Computación e Informática

Análisis e interpretación: Algoritmos recursivos de búsqueda y ordenamiento

Ayrton Morrison
Iván Gallardo
Pablo Gómez
Ingeniería Civil en Computación e Informática

Docente: MSc. Jackeline Aldridge
Ramo: Diseño de Algoritmos

RESUMEN

Este informe presenta el desarrollo de un sistema en lenguaje C para la gestión de información de deportistas, capaz de generar, almacenar, ordenar y buscar registros en archivos CSV. El proyecto se basa en la estrategia de *Divide y Vencerás*, implementando y analizando distintos algoritmos de ordenamiento, búsqueda y selección.

En particular, se implementaron los algoritmos de ordenamiento *Merge Sort* en su versión clásica y optimizada mediante *Insertion Sort*, además de *Quick Sort* utilizando el esquema de partición de Lomuto con distintas variantes de pivote: último elemento, primer elemento, pivote aleatorio y mediana de tres. Asimismo, se desarrollaron algoritmos de búsqueda como búsqueda binaria recursiva, búsqueda exponencial y búsqueda por interpolación, junto con una variante de búsqueda binaria para rangos. También se implementó el algoritmo de selección *Quick Select* para obtener el k-ésimo mejor deportista según puntaje.

El sistema incorpora funcionalidades prácticas como ordenamiento de deportistas según distintos criterios, búsqueda eficiente de registros, generación de rankings y obtención del mejor deportista en una posición determinada. Además, se realizaron benchmarks experimentales para evaluar el comportamiento de los algoritmos en distintos tamaños de entrada y escenarios de ejecución, considerando mejor caso, peor caso y caso promedio. Los resultados obtenidos permitieron comparar empíricamente el rendimiento de las distintas implementaciones y variantes estudiadas, contrastando además su comportamiento con el análisis teórico de complejidad temporal.

ÍNDICE GENERAL

ÍNDICE DE FIGURAS

INTRODUCCIÓN

"**Divide y Vencerás**" es un paradigma de programación, que consiste en un conjunto de técnicas algorítmicas en las que primero se divide el problema a resolver en subproblemas de menor tamaño y de la misma naturaleza, luego se resuelven los subproblemas de manera independiente y recursiva, hasta llegar a un caso base (que es un caso tan trivial que detiene la recursión), y finalmente se combinan los resultados para llegar a la solución final.

En el presente informe, se analizará empíricamente la complejidad algorítmica de varios algoritmos que utilizan este paradigma de programación, y además, se evaluará el impacto que tiene cada variante de cada algoritmo.

Los algoritmos que serán objeto de este análisis involucrarán tanto algoritmos de ordenamiento, como algoritmos de búsqueda, como también un algoritmo de selección.

Estos algoritmos son:

Ordenamiento

- *Merge Sort*.
- *Merge-Insertion Sort* (versión de *Merge Sort* que incluye *Insertion Sort* cuando se llegan a tener subarreglos de tamaño pequeño).
- *Quick Sort* con pivote en la primera posición.
- *Quick Sort* con pivote en la última posición.
- *Quick Sort* con pivote en una posición aleatoria.
- *Quick Sort* escogiendo al pivote como resultado de una mediana entre 3 elementos.

Búsqueda

- Búsqueda binaria clásica.
- Búsqueda binaria para rangos.
- Búsqueda exponencial.
- Búsqueda por interpolación.

Selección

- *Quick Select*.

Todo lo anterior se encuentra enmarcado en un sistema de gestión de deportistas desarrollado previamente, el cual trabaja con registros que contienen información como ID, nombre, equipo, puntaje y cantidad de competencias disputadas. Sobre este conjunto de datos, se implementaron distintas funcionalidades prácticas, tales como el ordenamiento de deportistas según distintos criterios, la búsqueda eficiente de registros, la obtención del k-ésimo mejor deportista mediante algoritmos de selección, y la generación de rankings basados en puntaje. Además, se realizaron *benchmarks* experimentales utilizando distintos tamaños de entrada, con el objetivo de comparar el comportamiento y el rendimiento de los algoritmos implementados en escenarios de mejor caso, peor caso y caso promedio.

OBJETIVOS

2.1. Objetivo Principal

2.2. Objetivos Secundarios

MARCO TEÓRICO

DESARROLLO

4.1. Diseño general del sistema

El sistema fue diseñado para gestionar información de deportistas de forma simple y modular, permitiendo generar datos, cargarlos desde archivos CSV y aplicar sobre ellos distintas operaciones de ordenamiento, búsqueda y análisis experimental. Para facilitar su desarrollo y comprensión, la implementación se organizó en componentes que separan la representación de los datos, la generación y carga de registros, y las funcionalidades principales ofrecidas por el programa.

4.1.1. Estructura de datos de los deportistas

La estructura de datos utilizada para representar a cada deportista se definió como una estructura con campos específicos para almacenar distinta información; estos campos corresponden a:

- **ID:** Identificador único del deportista.
- **Nombre:** Nombre completo del deportista.
- **Equipo:** Equipo al que pertenece.
- **Puntaje:** Puntuación total acumulada.
- **Competencias:** Número de competencias en las que ha participado.

Cada parámetro permite almacenar información, la cual puede ser utilizada para ordenar o buscar a los deportistas según distintos criterios. Esta estructura se implementó utilizando un **struct** en C, lo que facilita su manejo y acceso a los campos correspondientes durante las operaciones de ordenamiento y búsqueda.

4.1.2. Generación y carga de datos

La generación de datos de los parámetros asociados a deportistas se realizó mediante distintas funciones, las cuales permiten crear u obtener estos registros de manera pseudoaleatoria. La generación de un nombre se realizó permitiendo seleccionar aleatoriamente letras de un conjunto predefinido, escogiendo un tamaño de 3 a 8 caracteres. Para el equipo, se seleccionó aleatoriamente entre un conjunto específico de equipos de Esports. El ID se generó en forma incremental, asegurando la unicidad de cada registro. Por otro lado, el puntaje y la cantidad de competencias se generaron utilizando funciones de generación de números aleatorios dentro de rangos predefinidos. Esta metodología permitió crear un conjunto de datos variado y representativo para probar las funcionalidades del sistema. Posterior a la creación de los registros, estos se desordenaron utilizando el algoritmo de Fisher-Yates para garantizar que los datos no se encuentren en un orden específico, lo que es fundamental para evaluar correctamente el desempeño de los algoritmos de ordenamiento y búsqueda implementados. Finalmente, los registros generados se almacenaron en un archivo CSV, lo que facilita su posterior carga y manipulación dentro del programa.

4.2. Algoritmos de ordenamiento

Existen muchísimos algoritmos de ordenamiento, desde algunos muy eficientes, que pueden llegar a ser de una complejidad temporal de apenas $O(n)$, como *Radix Sort*, hasta algunos extremadamente ineficientes, que pueden llegar incluso a ser de $O(n \times n!)$, como lo es el caso de *Bogo Sort*, pero en este trabajo nos centramos solamente en *Merge Sort* y *Quick Sort*, que son algoritmos relativamente eficientes y que emplean el principio de "**Divide y Vencerás**".

4.2.1. Merge Sort

Merge Sort, también conocido como "Ordenamiento por Mezcla", es un algoritmo de ordenamiento recursivo, que en cada paso va dividiendo el arreglo en 2 subarreglos de aproximadamente la mitad del tamaño anterior, y eso lo hace hasta que todos los subarreglos sean de tamaño 1. Cuando eso sucede, se comienzan a combinar pares de subarreglos adyacentes, recorriéndolos linealmente para fusionarlos en el orden correcto, hasta que finalmente todo el arreglo queda ordenado.

En base al principio en el que se fundamenta *Merge Sort*, es posible deducir su complejidad temporal mediante la siguiente ecuación de recurrencia:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

Esto se debe a que el problema se divide recursivamente en 2 subproblemas de tamaño $n/2$, mientras que el costo adicional corresponde al proceso de fusión de los subarreglos, el cual requiere recorrer linealmente los elementos para ubicarlos en el orden correcto, por lo que dicho costo es de complejidad lineal.

Para resolver la ecuación de recurrencia, se puede emplear el método de iteración. Considerando una constante k para representar el costo lineal, se tiene:

$$T(n) = 2T\left(\frac{n}{2}\right) + kn$$

A partir de esto, la expresión equivalente para $T\left(\frac{n}{2}\right)$ sería:

$$T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{4}\right) + k\left(\frac{n}{2}\right)$$

Entonces, reemplazando en la ecuación original, se obtiene:

$$T(n) = 2\left(2T\left(\frac{n}{4}\right) + k\left(\frac{n}{2}\right)\right) + kn$$

Simplificando:

$$T(n) = 4T\left(\frac{n}{4}\right) + 2kn$$

Ahora, realizando otra iteración, se tiene que:

$$T\left(\frac{n}{4}\right) = 2T\left(\frac{n}{8}\right) + k\left(\frac{n}{4}\right)$$

Reemplazando nuevamente:

$$T(n) = 4\left(2T\left(\frac{n}{8}\right) + k\left(\frac{n}{4}\right)\right) + 2kn$$

$$T(n) = 8T\left(\frac{n}{8}\right) + 3kn$$

Continuando de esta forma, se puede generalizar que:

$$T(n) = 2T\left(\frac{n}{2}\right) + kn = 4T\left(\frac{n}{4}\right) + 2kn = 8T\left(\frac{n}{8}\right) + 3kn = 2^i T\left(\frac{n}{2^i}\right) + ikn$$

La recursión finaliza cuando se alcanza el caso base, es decir, cuando los subarreglos tienen tamaño 1. Por lo tanto, debe cumplirse que:

$$\frac{n}{2^i} = 1$$

Despejando:

$$i = \log n$$

Reemplazando esto en la expresión general obtenida anteriormente:

$$T(n) = 2^{\log n} T(1) + \log n kn$$

Como $2^{\log n} = n$, entonces:

$$T(n) = nT(1) + kn \log n$$

Finalmente, considerando que $T(1)$ y k son constantes, se concluye que:

$$T(n) \in O(n \log n)$$

Por lo tanto, *Merge Sort* posee complejidad temporal $O(n \log n)$ tanto en el mejor caso, como en el caso promedio y peor caso, ya que el arreglo siempre se divide de manera uniforme, independientemente del orden inicial de los elementos.

4.2.2. Merge-Insertion Sort

Merge-Insertion Sort corresponde a una variante optimizada de *Merge Sort*, cuyo objetivo es reducir el overhead recursivo que se produce al trabajar con subarreglos muy pequeños.

La idea principal consiste en que, cuando los subarreglos alcanzan un tamaño suficientemente pequeño, en lugar de continuar dividiéndolos recursivamente, se utiliza *Insertion Sort* para ordenarlos directamente.

Esto se hace debido a que *Insertion Sort*, a pesar de poseer complejidad $O(n^2)$ en el peor caso, tiene un rendimiento muy eficiente para arreglos de tamaño reducido, ya que posee un overhead muy bajo y además aprovecha favorablemente arreglos parcialmente ordenados.

De esta manera, *Merge-Insertion Sort* busca combinar las ventajas de ambos algoritmos: la eficiencia asintótica de *Merge Sort* para arreglos grandes, junto con la simplicidad y rapidez de *Insertion Sort* en subarreglos pequeños.

En términos de complejidad asintótica, esta variante sigue perteneciendo a $O(n \log n)$, aunque en la práctica puede presentar mejoras de rendimiento respecto a la versión clásica de *Merge Sort*, dependiendo del umbral utilizado para decidir cuándo aplicar *Insertion Sort*, que es uno de los aspectos que se analizará en el presente trabajo.

4.2.3. Quick Sort

Quick Sort es un algoritmo que consiste en seleccionar un pivote dentro del arreglo, y mediante comparaciones e intercambios al recorrer el arreglo, ir colocando los elementos que sean menores al pivote, a su izquierda, y los que sean mayores, a su derecha. Y luego con las particiones (subarreglos) que hayan quedado, volver a hacer lo mismo, hasta llegar al caso trivial que es cuando las particiones llegan a tener a lo más un elemento.

Existen varias formas de seleccionar el pivote en este algoritmo. Estas son:

- El primer elemento.
- El último elemento.
- Un elemento aleatorio.
- Mediante mediana de tres.

El mejor caso ocurre cuando el límite que separan las 2 particiones siempre ocurre prácticamente por la mitad. En ese caso, la complejidad temporal del algoritmo viene dada por la recurrencia:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

Esta recurrencia, es la misma que se obtuvo para *Merge Sort*, por lo cual el mejor caso de *Quick Sort* es el mismo que para *Merge Sort*, o sea $O(n \log(n))$.

El caso promedio ocurre cuando las particiones de *Quick Sort*, quedan un poquito desequilibradas pero no demasiado. Un caso promedio aproximado podría ser:

$$T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + O(n)$$

Esta recurrencia también tiene una solución $T(n) \in O(n \log n)$, por lo que la complejidad temporal para *Quick Sort* en caso promedio también es $O(n \log(n))$.

El peor caso ocurre por ejemplo cuando siempre la partición queda extremadamente desequilibrada, con 1 elemento en uno de los lados, y los otros $n-1$ del otro lado, por lo que la recurrencia para ese caso sería:

$$T(n) = T(n - 1) + O(n)$$

Resolviendo esa recurrencia, se llega a que en el peor caso, *Quick Sort* tiene una complejidad temporal de $O(n^2)$.

Un ejemplo de peor caso, es cuando el arreglo ya está ordenado y el método de seleccionar el pivote, es el primer elemento, o el último.

4.3. Algoritmos de búsqueda

4.3.1. Búsqueda binaria recursiva

La búsqueda binaria recursiva es un algoritmo de búsqueda, que al igual que los otros que se implementaron, utiliza el paradigma de "**Divide y Vencerás**". Funciona solo si el arreglo ya fue ordenado previamente. La lógica del algoritmo consiste en comparar el elemento buscado con el elemento ubicado en la posición central del arreglo. Si ambos elementos coinciden, la búsqueda finaliza exitosamente. En caso contrario, si el elemento central es mayor que el valor buscado, se descarta la mitad superior del arreglo y la búsqueda continúa únicamente sobre la primera mitad. Por otro lado, si el elemento central es menor que el valor buscado, se descarta la mitad inferior y la búsqueda prosigue sobre la segunda mitad del arreglo. Este proceso se repite recursivamente hasta encontrar el elemento o hasta que el rango de búsqueda se vuelva vacío.

El peor caso para la búsqueda binaria ocurre cuando el elemento buscado no se encuentra en el arreglo, o cuando se encuentra en una de las últimas divisiones realizadas por el algoritmo. En este escenario, el algoritmo debe continuar reduciendo el espacio de búsqueda hasta llegar a un subarreglo vacío o de tamaño 1.

El caso promedio ocurre cuando el elemento buscado se encuentra luego de una cantidad intermedia de divisiones del arreglo. Debido a que el algoritmo reduce el espacio de búsqueda aproximadamente a la mitad en cada paso, el número de comparaciones sigue creciendo de forma logarítmica respecto al tamaño del arreglo.

Por lo tanto, tanto el caso promedio como el peor caso presentan una complejidad temporal de $O(\log n)$

4.3.2. Búsqueda exponencial

La búsqueda exponencial es un algoritmo de búsqueda que combina una fase de expansión exponencial con búsqueda binaria. Al igual que la búsqueda binaria, requiere que los datos se encuentren previamente ordenados.

La lógica del algoritmo consiste en revisar secuencialmente elementos ubicados en posiciones que crecen exponencialmente, es decir, en posiciones de la forma:

$$k = 2^i, \quad \forall i \in \mathbb{N}$$

mientras el valor revisado sea menor que el elemento buscado y el índice permanezca dentro de los límites del arreglo.

Si el algoritmo encuentra directamente el elemento buscado en alguna de estas posiciones, la búsqueda finaliza inmediatamente. En caso contrario, cuando encuentra un elemento mayor que el valor buscado, se determina un rango acotado entre la última potencia de 2 revisada y la actual, aplicando posteriormente búsqueda binaria sobre dicho intervalo.

También puede ocurrir que el elemento buscado sea mayor que todos los elementos del arreglo. En ese caso, la fase exponencial continúa aumentando el índice hasta alcanzar o sobrepasar el tamaño del arreglo. Cuando esto sucede, el límite superior de búsqueda se ajusta al último índice válido del arreglo antes de aplicar búsqueda binaria sobre el intervalo restante.

El peor caso ocurre cuando el elemento buscado se encuentra cerca del final del arreglo, o sea cuando es mayor que la mayoría de los elementos almacenados. En ese caso, el algoritmo debe recorrer sucesivamente todas las posiciones que correspondan a potencias de 2 dentro de los límites del arreglo antes de determinar el rango final de búsqueda y aplicar búsqueda binaria sobre dicho intervalo.

Debido a que la fase de expansión exponencial realiza a lo más $O(\log n)$ comparaciones, y posteriormente la búsqueda binaria también posee complejidad $O(\log n)$, la complejidad temporal total del algoritmo en el peor caso es $O(\log n)$

El caso promedio ocurre cuando el elemento buscado se encuentra en una posición intermedia del arreglo, de manera que la fase exponencial logra acotar relativamente rápido el intervalo donde podría encontrarse el dato. Posteriormente, la búsqueda binaria se aplica únicamente sobre dicho rango acotado. Al igual que en el peor caso, la complejidad temporal promedio de la búsqueda exponencial es $O(\log n)$.

4.3.3. Búsqueda por interpolación

La búsqueda por interpolación es un algoritmo de búsqueda que requiere que los datos se encuentren previamente ordenados. Su funcionamiento consiste en estimar la posición probable del elemento buscado, suponiendo que los datos se encuentran distribuidos de manera uniforme.

La posición estimada se calcula mediante la siguiente fórmula:

$$supposed_idx = start_idx + \frac{element - array[start_idx]}{array[end_idx] - array[start_idx]} \times (end_idx - start_idx)$$

A diferencia de la búsqueda binaria, que siempre revisa el elemento central del arreglo, la búsqueda por interpolación intenta aproximarse directamente a la posición donde probablemente se encuentre el dato buscado. Esto puede mejorar considerablemente el rendimiento cuando los datos están distribuidos uniformemente.

El peor caso para este algoritmo ocurre cuando los datos presentan una distribución muy desigual o altamente sesgada hacia alguno de los extremos del arreglo. Por ejemplo al tener algo así:

```
int array [] = {1, 2, 3, 4, 5, 1000, 10000};
```

Si se desea buscar el elemento 1000, la fórmula de interpolación estimará inicialmente una posición muy cercana al inicio del arreglo, debido a la gran diferencia entre los últimos elementos y el resto de los datos. Como consecuencia, las estimaciones realizadas serán poco precisas y el algoritmo reducirá muy lentamente el espacio de búsqueda, pudiendo degradarse hasta una complejidad temporal de $O(n)$.

El caso promedio ocurre cuando los datos están parcialmente uniformes y sin valores que se escapen demasiado de los demás. En ese caso la complejidad temporal es de apenas $O(\log(\log n))$.

CONCLUSIÓN

ANEXOS

6.1. Código fuente

6.2. Contribución por integrante